

# Teoria od podstaw v3

Wprowadzenie do obliczeń numerycznych, FEM, backendów, benchmarków i replayu poprawności dla osoby spoza informatyki

## Zakres dokumentu

- Teoria od podstaw

Wygenerowano: 2026-04-19T12:11:32.092581+02:00

Katalog roboczy: .../v3

# Spis treści

Sekcje ułożone w kolejności czytania i pracy eksperymentalnej.

## 1. Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

# Teoria od podstaw v3

## Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Plik źródłowy: docs/TEORIA\_OD\_PODSTAW.md

Sekcja dokumentacji v3 przygotowana do pracy eksperymentalnej i pisania rozprawy.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Ten dokument jest napisany dla osoby, która:

- nie jest informatykiem,
- nie programuje zawodowo,
- nie zajmuje się architekturą komputerów,
- ale chce dobrze zrozumieć, co jest badane w v3, po co ten system powstał i dlaczego ma sens naukowy.

To nie jest instrukcja techniczna. To jest spokojne wprowadzenie do idei projektu.

Można go czytać jak mapę pojęć: od najprostszych intuicji do bardziej specjalistycznych warstw eksperymentu.

## 1. Po co w ogóle istnieje ten projekt

Najprostsza odpowiedź brzmi:

- chcemy zrozumieć, dlaczego to samo obliczenie może działać szybciej albo wolniej na różnych komputerach,
- ale jednocześnie chcemy mieć pewność, że porównujemy to samo obliczenie, a nie dwa podobne, lecz jednak różne programy.

To bardzo ważne, bo w badaniach wydajności łatwo popełnić błąd interpretacyjny.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Można zobaczyć, że:

- jeden system jest szybszy,
- drugi jest wolniejszy,
- wykres wygląda przekonująco,

ale w rzeczywistości:

- dane wejściowe były inne,
- wynik końcowy nie był sprawdzony,
- albo program wykonywał trochę inną sekwencję obliczeń.

W takim przypadku szybki wynik nie musi jeszcze oznaczać, że porównanie jest uczciwe.

Dlatego v3 nie bada tylko czasu wykonania.

Bada trzy rzeczy jednocześnie:

1. wydajność - jak szybko system wykonuje pracę,
2. poprawność - czy wynik obliczeń jest zgodny z referencją,
3. interpretację - dlaczego uzyskano taki wynik i co on mówi o architekturze sprzętu.

To właśnie odróżnia platformę badawczą od zwykłego zestawu testów wydajnościowych.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

## 2. Czym w ogóle jest obliczenie numeryczne

W naukach technicznych bardzo często chcemy odpowiedzieć na pytania typu:

- jak rozchodzi się ciepło,
- jak płynie ciecz,
- jak odkształca się materiał,
- jak zmienia się pole w pewnym obszarze,
- jak zachowuje się układ fizyczny w czasie.

Takie zjawiska opisuje się zwykle matematyką, najczęściej w postaci równań różniczkowych.

W teorii brzmi to elegancko, ale w praktyce bardzo rzadko da się rozwiązać taki problem dokładnie "na kartce" dla rzeczywistej geometrii i rzeczywistych warunków.

Wtedy wchodzi obliczenie numeryczne.

To znaczy:

- zamiast próbować znaleźć idealne rozwiązanie wzorem,
- budujemy przybliżenie,
- dzielimy problem na mniejsze części,
- i pozwalamy komputerowi policzyć wynik krok po kroku.

Można to porównać do mierzenia bardzo skomplikowanej linii brzegowej.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Nie da się jej jednym ruchem opisać idealnie, więc dzielimy ją na małe odcinki i sumujemy przybliżenia.

W obliczeniach numerycznych robimy coś podobnego, tylko na dużo większą skalę i w dużo bardziej złożonych problemach.

## 3. Model, symulacja i wynik - trzy różne rzeczy

Warto od razu odróżnić trzy pojęcia.

### 3.1. Model

Model to opis badanego zjawiska.

Na przykład:

- model przewodzenia ciepła,
- model przepływu,
- model odkształcenia.

Model mówi, co opisujemy.

### 3.2. Symulacja

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Symulacja to wykonanie obliczenia tego modelu na komputerze.

Symulacja mówi, jak komputer próbuje policzyć model.

## 3.3. Wynik

Wynik to konkretna liczba, pole, macierz, wykres lub zbiór danych otrzymany po symulacji.

Wynik odpowiada na pytanie: co wyszło.

To rozróżnienie jest ważne, bo w v3 nie badamy samej fizyki modelu.

Badamy także:

- jak zorganizowano obliczenie,
- jak sprzęt radzi sobie z tym obliczeniem,
- i czy wynik końcowy pozostał zgodny mimo zmiany backendu lub architektury.

## 4. Czym jest metoda elementów skończonych (FEM)

Jedną z najważniejszych metod stosowanych w obliczeniach numerycznych jest metoda elementów skończonych, czyli FEM.

Jej intuicja jest prosta.



# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Zamiast patrzeć na cały obszar jako na jeden ogromny i trudny problem, dzieli się go na wiele małych fragmentów.

Te fragmenty nazywamy elementami.

Można to porównać do:

- pocięcia skomplikowanego kształtu na małe kawałki,
- pokrycia powierzchni drobną siatką,
- zbudowania większego obrazu z wielu małych płytek.

Każdy mały kawałek łatwiej opisać matematycznie.

Potem wyniki z tych małych kawałków składa się w większą całość.

## 5. Co składa się na jeden element FEM

### 5.1. Element

Element to mały fragment obszaru obliczeniowego.

W tej pracy interesują nas między innymi elementy pryzmatyczne.

Nie trzeba znać ich dokładnej geometrii, żeby zrozumieć ideę: to po prostu jeden z typów „cegielek”, z których złożona jest siatka.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

## 5.2. Węzły

Każdy element ma charakterystyczne punkty, zwane węzłami.

Można myśleć o nich jak o punktach zaczepienia, w których opisujemy stan rozwiązania.

## 5.3. Funkcje kształtu

Żeby opisać, co dzieje się wewnątrz elementu, używa się funkcji kształtu.

To matematyczne funkcje, które pozwalają wyznaczyć zachowanie wnętrza elementu na podstawie wartości w jego węzłach.

W uproszczeniu:

- nie liczymy wszystkiego w nieskończenie wielu punktach,
- tylko budujemy inteligentne przybliżenie wewnątrz elementu.

## 5.4. Punkty całkowania

W wielu obliczeniach FEM trzeba policzyć wkład z wnętrza elementu.

Zamiast robić to dokładnie w każdym punkcie, wybiera się specjalne punkty pomocnicze, w których obliczenie jest wykonywane.

To są punkty całkowania albo punkty Gaussa.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Można je traktować jak mądrze dobrane punkty kontrolne.

## 6. Czym są macierz i wektor w tym projekcie

Dla osoby spoza matematyki te słowa potrafią brzmieć odstraszaająco, ale intuicja jest dość prosta.

### 6.1. Wektor

Wektor to uporządkowana lista liczb.

Można go traktować jak kolumnę wartości.

W FEM wektor często opisuje:

- wymuszenia,
- źródła,
- wpływy zewnętrzne,
- albo część stanu układu.

### 6.2. Macierz

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Macierz to prostokątna tabela liczb.

Każda liczba mówi, jak jeden fragment układu wpływa na inny.

Najprościej:

- wektor to lista,
- macierz to tabela zależności.

## 6.3. Lokalna macierz i lokalny wektor

Dla pojedynczego elementu komputer wylicza zwykle:

- lokalną macierz,
- lokalny wektor prawej strony.

Intuicyjnie:

- lokalna macierz opisuje zależności wewnątrz małego fragmentu,
- lokalny wektor opisuje, jakie działają tam wymuszenia lub źródła.

Dopiero potem te lokalne obiekty składa się w większy układ globalny.

## 7. Dlaczego jeden mały element jest ważny dla wydajności

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Bo takich elementów w realnym problemie może być bardzo dużo.

To oznacza, że jeśli pojedynczy element liczy się trochę szybciej, to po przemnożeniu przez tysiące lub miliony elementów zysk staje się bardzo duży.

Dlatego nawet pozornie niewielkie różnice w organizacji kernela FEM mają znaczenie praktyczne.

## 8. Co to jest kernel

W tym projekcie bardzo często pojawia się słowo kernel.

Najprościej można o nim myśleć jak o małym, wyspecjalizowanym fragmencie programu, który wykonuje powtarzalną pracę obliczeniową.

Na GPU kernel jest uruchamiany równolegle w wielu kopiach naraz.

Czyli:

- kernel to nie cały program,
- kernel to najważniejszy fragment roboczy, w którym dzieje się właściwe liczenie.

W kontekście v3 kernel FEM to fragment odpowiedzialny za policzenie wkładów elementowych.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

## 9. CPU i GPU - intuicja bez żargonu

### 9.1. CPU

CPU jest bardziej uniwersalne.

Dobrze radzi sobie z:

- złożoną logiką,
- wieloma różnymi zadaniami,
- nieregularnym sterowaniem,
- sytuacjami, w których trzeba często podejmować decyzje.

Można o nim myśleć jak o małej grupie bardzo wszechstronnych specjalistów.

### 9.2. GPU

GPU zostało zaprojektowane do masowego wykonywania wielu podobnych operacji równocześnie.

Można o nim myśleć jak o bardzo dużej grupie pracowników, z których każdy wykonuje prostszy fragment tej samej pracy.

GPU jest bardzo silne wtedy, gdy:

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

- pracę można dobrze podzielić,
- wiele operacji wygląda podobnie,
- dane są ułożone w sposób przyjazny dla pamięci,
- narzut organizacyjny nie dominuje nad samym liczeniem.

## 10. Co to jest pamięć i dlaczego jest tak ważna

Wydajność obliczeń nie zależy tylko od tego, jak szybko układ liczy.

Bardzo często zależy od tego, jak szybko potrafi pobrać dane i zapisać wynik.

Można to porównać do kuchni:

- nawet najlepszy kucharz nic nie zrobi szybko, jeśli co chwilę musi biec do odległego magazynu po składniki,
- jeśli składniki są dobrze przygotowane i ułożone, praca przyspiesza.

W świecie obliczeń komputerowych bardzo często to właśnie ruch danych, a nie samo liczenie, staje się prawdziwym ograniczeniem.

## 11. Co to jest backend

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Backend w tym projekcie oznacza praktyczny sposób wykonania obliczenia na danej platformie.

To warstwa, która odpowiada na pytania:

- jak przekazać dane do urządzenia,
- jak uruchomić kernel,
- jak zbudować kod dla danej platformy,
- jak odebrać wynik.

Przykładowe backendy w v3 to:

- cpu,
- opencl,
- metal.

To ważne rozróżnienie.

Ten sam problem matematyczny może być uruchomiony przez różne backendy.

Wtedy logika zadania pozostaje podobna, ale sposób wykonania i koszty sprzętowe mogą być zupełnie inne.

## 12. Co to znaczy, że obliczenie jest równoległe

Obliczenie równoległe to takie, w którym wiele fragmentów pracy wykonuje się jednocześnie.



# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Jeśli zadanie da się podzielić na dużą liczbę podobnych fragmentów, można je rozdać wielu jednostkom wykonawczym naraz.

To właśnie jest jedna z głównych przewag GPU.

Ale sama równoległość nie gwarantuje sukcesu.

Trzeba jeszcze zadbać o:

- podział pracy,
- organizację pamięci,
- odpowiednią wielkość grup roboczych,
- ograniczenie zbędnych synchronizacji.

## 13. Co to jest workgroup

Na GPU praca jest organizowana w grupy robocze.

Można je traktować jak małe zespoły pracowników wykonujących pokrewny fragment zadania.

Sposób zorganizowania tych zespołów wpływa na:

- współdzielenie pamięci,
- koszt komunikacji,
- możliwość ponownego użycia danych,

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

- narzut sterowania.

Dlatego parametry grup roboczych często istotnie wpływają na wynik benchmarku.

## 14. Czym jest benchmark

Benchmark to uporządkowany test, który ma coś zmierzyć.

W tym projekcie benchmark nie jest po prostu pojedynczym programem.

To raczej cała procedura pomiarowa, która odpowiada na pytanie:

- jak dany system zachowuje się w określonych warunkach?

Benchmark może mierzyć:

- czas,
- przepustowość,
- liczbę operacji na sekundę,
- zużycie energii,
- stabilność,
- zgodność wyniku.

## 15. Czym jest mikrobenchmark

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Mikrobenchmark to bardzo mały, celowo uproszczony benchmark.

Jego zadaniem nie jest udawanie całej aplikacji, tylko zbadanie jednego konkretnego mechanizmu.

Można to porównać do medycyny.

Zamiast od razu opisywać cały organizm, wykonuje się konkretne badanie, które sprawdza jeden wybrany aspekt.

Mikrobenchmarki w v3 badają między innymi:

- przepustowość pamięci,
- opóźnienie dostępu do pamięci,
- przepustowość obliczeniową,
- wrażliwość na wzorzec odczytu i zapisu,
- skutki różnych organizacji pracy.

## 15.1. Dlaczego mikrobenchmarki są potrzebne

Bo same czasy końcowej aplikacji są zbyt mało wyjaśniające.

Jeśli widzimy, że jeden backend jest szybszy, a drugi wolniejszy, to bez dodatkowej wiedzy nie wiemy:

- czy ogranicza go pamięć,

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

- czy ogranicza go liczenie,
- czy problemem jest organizacja danych,
- czy narzut uruchomienia,
- czy niewłaściwy podział pracy.

Mikrobenchmarki tworzą więc warstwę diagnostyczną.

## 15.2. Dlaczego same mikrobenchmarki nie wystarczają

Bo są zbyt małe i zbyt czyste.

Nie reprezentują w pełni realistycznej aplikacji.

Dlatego v3 nie kończy się na mikrobenchmarkach. Traktuje je jako pierwszą warstwę wyjaśniającą, a następnie sprawdza, czy te same obserwacje mają sens również dla realistycznego kernela FEM.

## 16. Najważniejsze pojęcia wydajnościowe

### 16.1. Przepustowość pamięci

To odpowiedź na pytanie:

- jak dużo danych można przenieść w jednostce czasu?

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Jeśli obliczenie ciągle czeka na dane z pamięci, to nawet bardzo silny układ obliczeniowy nie pokaże pełnej mocy.

## 16.2. Opóźnienie

To odpowiedź na pytanie:

- jak długo trzeba czekać na pojedynczą reakcję lub pojedynczy dostęp?

Można mieć wysoką przepustowość, ale wciąż odczuwać koszt pojedynczych operacji.

## 16.3. Throughput obliczeniowy

To odpowiedź na pytanie:

- ile pracy obliczeniowej można wykonać w jednostce czasu?

Jeśli problem jest „bogaty obliczeniowo”, ten parametr staje się bardzo ważny.

## 16.4. Wąskie gardło

Wąskie gardło to element systemu, który realnie ogranicza szybkość całości.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Może nim być:

- pamięć,
- przepustowość zapisu,
- zbyt mały równoległy podział pracy,
- narzut uruchomienia,
- albo samo liczenie.

Jednym z głównych celów v3 jest właśnie ustalenie, gdzie to wąskie gardło się znajduje.

## 16.5. Roofline

Roofline to model, który pomaga odróżnić dwa główne rodzaje ograniczeń:

- ograniczenie przez pamięć,
- ograniczenie przez moc obliczeniową.

Nie trzeba znać całej matematyki tego modelu, żeby uchwycić sens.

Roofline to po prostu sposób patrzenia na problem przez pytanie:

- czy bardziej brakuje nam danych,
- czy bardziej brakuje nam mocy liczenia?

## 17. Dlaczego kernel może mieć różne opcje organizacyjne

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Realistyczny kernel FEM można wykonać na różne sposoby, nie zmieniając jego celu matematycznego.

To trochę jak gotowanie według tego samego przepisu, ale z inną organizacją kuchni:

- można składniki pobierać z magazynu za każdym razem,
- można część z nich przygotować wcześniej,
- można trzymać je bliżej siebie,
- można coś przechowywać chwilowo, żeby użyć ponownie,
- można coś policzyć jeszcze raz zamiast to przechowywać.

W v3 i w warstwie referencyjnej pojawiają się więc różne opcje organizacyjne.

Najważniejsze z nich to:

## 17.1. coal\_read

To dotyczy sposobu odczytu danych.

Intuicyjnie:

- czy wiele wątków pobiera dane w uporządkowany sposób,
- czy raczej każdy sięga gdzie indziej.

Uporządkowany odczyt zwykle bardziej odpowiada sprzętowi.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

## 17.2. coal\_write

To samo, ale dla zapisu wyników.

Zorganizowany zapis jest zwykle łatwiejszy dla pamięci niż zapis chaotyczny.

## 17.3. compute\_all\_shape\_fun\_der

To decyzja typu:

- czy pewne wartości lepiej policzyć na miejscu,
- czy lepiej odczytać je z pamięci.

To klasyczny kompromis:

- więcej liczenia,
- albo więcej ruchu pamięciowego.

Różne architektury mogą preferować różne strony tego kompromisu.

## 17.4. workspace\_\*

To dotyczy użycia dodatkowej pamięci roboczej.



# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Można ją porównać do wspólnego notesu, z którego korzysta mały zespół.

Zalety:

- można coś przechować lokalnie i użyć ponownie.

Wada:

- taki notes ma ograniczoną pojemność,
- a korzystanie z niego też kosztuje.

## 17.5. padding

Padding to celowe dodawanie odstępów lub pustych miejsc w danych.

Brzmi dziwnie, ale czasem pomaga sprzętowi lepiej organizować pamięć.

To trochę jak zostawienie luzu na półce, żeby szybciej wyjmować przedmioty.

## 18. Czym jest fem\_option\_validation

To bardzo ważna warstwa v3.

Jej rola jest pośrednia między:

- prostymi mikrobenchmarkami,

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

- a pełnym, realistycznym kernelem FEM.

Dlaczego jest potrzebna?

Bo mikrobenchmarki są zbyt proste, a pełny kernel jest na tyle złożony, że trudniej jednoznacznie powiedzieć, co naprawdę zadecydowało o wyniku.

fem\_option\_validation działa jak most.

Sprawdza:

- jak backend reaguje na wybrane, kontrolowane zmiany,
- które są bardzo bliskie temu, co dzieje się później w realistycznym kernelu.

To znaczy:

- nie jest to jeszcze pełna aplikacja,
- ale nie jest to też już tylko prymitywny test jednej operacji.

W praktyce ta warstwa pomaga odpowiedzieć na pytanie:

- czy obserwacja z mikrobenchmarku pozostaje sensowna, kiedy obliczenie zaczyna przypominać prawdziwy kernel FEM?

## 19. Czym jest kampania referencyjna exact

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Warstwa `reference_exact` to referencyjny przebieg oparty o oryginalną implementację OpenCL.

Ważne: słowo `exact` nie oznacza tutaj matematycznej doskonałości w sensie absolutnym.

Oznacza raczej:

- możliwie wierne odtworzenie referencyjnej, historycznej ścieżki wykonania.

Ta warstwa daje nam:

- referencyjny sposób liczenia,
- referencyjne dane wejściowe dla kernela,
- referencyjny wynik,
- referencyjne metadane uruchomienia.

To jest coś w rodzaju wzorca laboratoryjnego.

## 20. Dlaczego nie wystarczy tylko ten sam plik problemu

Na pierwszy rzut oka mogłoby się wydawać, że jeśli dwa backendy dostaną ten sam plik siatki i ten sam plik problemu, to dostają te same dane.

W praktyce to jeszcze za mało.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Po drodze system buduje bardziej szczegółowe bufory runtime, czyli dane przygotowane już konkretnie dla kernela.

To właśnie one decydują o tym, co kernel naprawdę dostaje do policzenia.

Dlatego v3 zamraża nie tylko plik wysokiego poziomu, ale rzeczywiste dane wejściowe dla kernela.

To jest warunek uczciwego porównania.

## 21. Czym jest replay na zamrożonych danych wejściowych

To jedna z najważniejszych idei całego systemu.

Frozen-input replay oznacza:

- bierzemy dokładnie te same dane wejściowe do obliczenia,
- zapisujemy je,
- a potem uruchamiamy je na innym backendzie.

Czyli:

- nie generujemy nowych danych,
- nie liczymy „podobnego” przypadku,
- tylko odtwarzamy to samo wejście.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Dzięki temu możemy sprawdzić:

- czy inny backend liczy to samo,
- a nie tylko czy daje podobny czas.

## 21.1. Po co to jest potrzebne

Bo bez tego zawsze można postawić zarzut:

- może jeden backend był szybszy, ale tak naprawdę liczył coś trochę innego.

Replay zamyka ten problem.

## 22. Czym jest expected output

Expected output to oczekiwany, referencyjny wynik obliczenia.

Jeśli mamy:

- zamrożone dane wejściowe,
- oraz zapisany wynik referencyjny,

możemy sprawdzić, czy backend docelowy daje ten sam wynik w granicach dopuszczalnej różnicy numerycznej.

## Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

To właśnie jest warstwa walidacji poprawności.

### 23. Dlaczego wyniki nie zawsze muszą być bit po bicie identyczne

Komputery liczą na liczbach zmiennoprzecinkowych.

To oznacza, że bardzo drobne różnice mogą pojawiać się naturalnie, nawet jeśli obliczenie jest logicznie to samo.

Przyczyny mogą być różne:

- inna kolejność operacji,
- inne optymalizacje kompilatora,
- inne wykorzystanie instrukcji sprzętowych,
- różne szczegóły wykonania na dwóch platformach.

Dlatego poprawności nie ocenia się wyłącznie przez pytanie:

- czy każdy bit jest identyczny?

Częściej ocenia się przez:

- maksymalną różnicę bezwzględną,
- błąd średniokwadratowy,

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

- liczbę przypadków mieszczących się w tolerancji.

To jest podejście uczciwe naukowo.

## 24. Czym jest profiler

Profiler to narzędzie, które zagląda do środka wykonania programu.

Jeśli benchmark odpowiada na pytanie:

- „jak szybko?”

To profiler pomaga odpowiedzieć:

- „dlaczego akurat tak szybko albo tak wolno?”.

Profiler może pokazywać między innymi:

- ile czasu zajął dany kernel,
- czy obliczenie czekało na pamięć,
- czy sprzęt był dobrze wykorzystany,
- czy problemem był odczyt, zapis albo organizacja pracy.

To czyni profilery bardzo ważnym łącznikiem między teorią a praktyką.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

## 25. Czym jest profiler\_correlation

To warstwa, która spina trzy rodzaje wiedzy:

- wiedzę z mikrobenchmarków,
- wiedzę z fem\_option\_validation,
- wiedzę z realistycznego kernela i raportów profilerowych.

Można to rozumieć tak:

1. mikrobenchmarki mówią, co architektura lubi albo czego nie lubi,
2. walidacja FEM sprawdza to na bardziej realistycznych próbach,
3. profiler sprawdza, czy w prawdziwym kernelu widać zgodny mechanizm,
4. raport korelacyjny próbuje połączyć te trzy warstwy w jedną interpretację.

To jest bardzo wartościowe, bo pozwala przejść od:

- „tak wyszło w pomiarze”

do:

- „wiemy, co najprawdopodobniej spowodowało ten wynik”.



# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

## 26. Czym są provenance i hashe

W badaniach eksperymentalnych bardzo ważne jest pytanie:

- czy da się odtworzyć ten sam wynik?

Do tego potrzebne są informacje o pochodzeniu artefaktów, czyli provenance.

W v3 oznacza to między innymi:

- jaki plik powstał,
- kiedy powstał,
- w jakim środowisku,
- z jakiego uruchomienia,
- i czy na pewno nie został po drodze zmieniony.

### 26.1. Hash

Hash można traktować jak cyfrowy odcisk palca pliku.

Jeśli plik się zmieni, jego hash też się zmieni.

Dzięki temu można sprawdzić:

- czy dwa pliki są dokładnie tym samym artefaktem,

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

- czy wynik jest rzeczywiście tym, na który się powołujemy.

W kontekście doktoratu bardzo wzmacnia to wiarygodność pracy.

## 27. Dlaczego powtórzenia i statystyka są konieczne

Komputer nie pracuje w idealnej próżni.

Na wynik czasu mogą wpływać:

- inne procesy w tle,
- nagrzanie sprzętu,
- chwilowe zmiany obciążenia,
- zachowanie systemu operacyjnego,
- drobna niestabilność uruchomień.

Dlatego pojedynczy przebieg prawie nigdy nie powinien być jedyną podstawą wniosku.

Potrzebne są:

- powtórzenia,
- średnie,
- rozrzut wyników,
- czasem przedziały niepewności.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

To pozwala odróżnić:

- prawdziwy efekt,
- od jednorazowego przypadku.

## 28. Jak czytać wykresy w takim projekcie

Dla osoby spoza informatyki wykresy wydajności często wyglądają groźnie, ale zwykle odpowiadają na dość proste pytania.

### 28.1. Wykres słupkowy

Najczęściej pokazuje porównanie kilku wariantów.

Pytanie brzmi wtedy zwykle:

- który wariant był szybszy,
- który zużył mniej energii,
- który miał mniejszy błąd.

### 28.2. Wykres trendu

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Pokazuje, jak wynik zmienia się wraz ze wzrostem parametru.

Na przykład:

- liczby elementów,
- rozmiaru grupy roboczej,
- liczby punktów całkowania.

## 28.3. Wykres korelacyjny

Pokazuje, czy dwie rzeczy „idą razem”.

Na przykład:

- czy backend, który ma dobrą przepustowość pamięci, rzeczywiście lepiej radzi sobie z wariantami opartymi o uporządkowany odczyt.

W badaniach ważne jest jednak, żeby nie mylić korelacji z dowodem przyczyny.

Korelacja wspiera interpretację, ale nie zastępuje całej metodologii.

## 29. Jak warstwy v3 układają się w jedną historię

Najprostsza mapa logiczna wygląda tak:

### 1. Mikrobenchmarki

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

- mówią, jakie są podstawowe cechy sprzętu.

## 2. Walidacja opcji FEM

- sprawdza, czy te cechy mają znaczenie dla wzorców bliskich rzeczywistemu kernelowi FEM.

## 3. Reference exact

- dostarcza referencyjnego sposobu liczenia i zamrożonych danych wejściowych.

## 4. Replay poprawności

- sprawdza, czy inny backend liczy to samo na tych samych danych.

## 5. Korelacja profilerowa

- scala wiedzę z poprzednich warstw i pomaga wyjaśniać wyniki.

To oznacza, że v3 nie jest tylko zbiorem testów.

To jest uporządkowana metodologia badawcza.

## 30. Co można twierdzić naukowo, a czego nie

### 30.1. Co można twierdzić mocno

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Jeśli:

- mikrobenchmarki wskazują określone ograniczenie,
- fem\_option\_validation pokazuje zgodny trend,
- profiler pokazuje zgodny mechanizm,
- replay potwierdza zgodność obliczeń,

to można bardzo mocno twierdzić, że:

- interpretacja architektoniczna ma sens,
- a porównanie backendów dotyczy rzeczywiście tego samego obliczenia.

## 30.2. Czego nie należy twierdzić zbyt mocno

Nie powinno się twierdzić, że:

- sam podobny czas oznacza poprawne obliczenie,
- sama zgodność wykresu wydajności oznacza zgodność matematyczną,
- sama natywna kampania wydajnościowa jest dowodem ścisłej zgodności 1:1 wobec referencji.

To byłyby zbyt mocne uproszczenia.

## 31. Dlaczego to ma sens pod doktorat

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Dla doktoratu ważne jest nie tylko to, że system działa.

Ważniejsze jest to, że:

- ma jasną logikę,
- rozdziela role poszczególnych warstw,
- daje możliwość kontroli poprawności,
- umożliwia interpretację wyników,
- i pozostawia artefakty, które można odtworzyć i zarchiwizować.

To czyni z v3 nie tylko narzędzie do uruchamiania testów, ale realną platformę badawczą.

## 32. Słownik najważniejszych pojęć

### Architektura

Sposób zbudowania i organizacji sprzętu komputerowego.

### Backend

Warstwa wykonawcza, która uruchamia obliczenie na konkretnej platformie.

### Benchmark

Uporządkowany test mierzący określone zachowanie systemu.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

## Replay poprawności

Uruchomienie tego samego obliczenia na tych samych zamrożonych danych wejściowych w celu sprawdzenia poprawności wyniku.

## Expected output

Referencyjny wynik, z którym porównuje się wynik backendu docelowego.

## FEM

Metoda elementów skończonych - sposób przybliżania rozwiązań problemów inżynierskich przez podział obszaru na małe elementy.

## Frozen inputs

Zamrożone dane wejściowe do kernela, zapisane tak, aby dało się odtworzyć dokładnie to samo obliczenie.

## GPU

Procesor wyspecjalizowany w masowym wykonywaniu wielu podobnych operacji równolegle.

## Hash



# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

Cyfrowy odcisk palca pliku.

## Kernel

Najważniejszy fragment obliczeniowy wykonywany wielokrotnie, często równolegle.

## Lokalna macierz

Macierz opisująca zależności wewnątrz pojedynczego elementu FEM.

## Mikrobenchmark

Mały benchmark badający jeden wybrany mechanizm sprzętowy lub wykonawczy.

## Profiler

Narzędzie pokazujące, gdzie program spędza czas i co go ogranicza.

## Provenance

Informacja o pochodzeniu artefaktu: skąd pochodzi, kiedy powstał i w jakim środowisku.

## Replay bundle

Pakiet danych potrzebnych do odtworzenia obliczenia na innym backendzie.

# Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

## Tolerancja

Dopuszczalny poziom drobnej różnicy numerycznej między dwoma wynikami.

## Węzeł

Charakterystyczny punkt elementu FEM, w którym opisuje się wartości rozwiązania.

## Workgroup

Grupa robocza w modelu równoległego wykonania na GPU.

## 33. Najkrótsze podsumowanie dla osoby spoza informatyki

Jeśli trzeba to powiedzieć w jednym zdaniu, najuczciwiej brzmi to tak:

- v3 służy do sprawdzania, dlaczego to samo obliczenie inżynierskie zachowuje się różnie na różnych komputerach, przy jednoczesnym pilnowaniu, żeby porównywane systemy rzeczywiście liczyły to samo.

A jeśli w dwóch zdaniach:

- mikrobenchmarki mówią, jakie są możliwości i ograniczenia sprzętu,

## Teoria od podstaw

Wyjaśnienie pojęć i logiki projektu dla osoby bez przygotowania informatycznego.

- a warstwy FEM, replayu i korelacji profilerowej sprawdzają, czy te same obserwacje pozostają prawdziwe dla realistycznego obliczenia.